



9주차 과제

⚙ Status	In progress
📁 Task type	

1. LORA

◆ 1. Abstract (무슨 문제 해결?)

- 대형 LLM fine-tuning 비용(메모리·파라미터) 너무 큼
- → Low-Rank Adaptation (LoRA) 제안
- 기존 weight는 고정하고 저차원(rank) 행렬만 학습
- 👉 파라미터 10,000배 감소 + 성능 유지/향상

◆ 2. Introduction (동기 & 문제)

- GPT-3 같은 모델:
 - fine-tuning 시 모델 복사 필요 (수백 GB)
- 기존 방법:
 - Adapter → 추론 느림
 - Prompt tuning → 성능 불안정 / 입력 길이 감소
- 핵심 문제:
 - 👉 효율 + 성능 동시에 만족 못함

◆ 3. Related Works (연구 흐름)

- Fine-tuning: 모든 파라미터 학습 (비효율)
 - Adapter: 작은 모듈 추가 (latency 증가)
 - Prompt tuning: 입력만 조정 (표현력 제한)
 - Low-rank 연구:
 - 모델이 **low intrinsic dimension** 가진다는 연구 존재
-

◆ 4. Method (핵심 아이디어 & 구조 ★)

💡 핵심 아이디어

👉 "가중치 변화 ΔW 는 사실 low-rank일 것"

📌 수식

$$W = W_0 + \Delta W = W_0 + BA$$

- W_0 : pretrained (freeze)
- $B \in \mathbb{R}^{d \times r}, A \in \mathbb{R}^{r \times k}$
- $r \ll d$

👉 forward:

$$h = W_0 x + BAx$$

📌 적용 위치

- Transformer의 **Attention (W_q, W_v)**에 적용

📌 특징

- 학습: A, B만
 - 추론: W_0 + BA로 합쳐서 사용 → **latency 없음**
-

◆ 5. Experiments (실험 & 결과)

주요 결과

- GLUE, GPT-2, GPT-3 모두:
 - **fine-tuning**과 동일 or 더 좋음
- GPT-3 (175B):
 - LoRA가 더 높은 성능 (Table 4)

효율성

- 파라미터: **0.01%** 수준
 - VRAM: **3배** 감소
 - 속도: **~25%** 향상
-

◆ 6. Discussion (발견 & 한계)

발견

- ΔW 는 실제로 **low-rank** 구조
- rank = 1~4도 충분한 경우 많음
- LoRA는:
 - 👉 기존 weight의 "덜 강조된 방향"을 강화

한계

- task마다 최적 rank 다름
- 여러 task batch 처리 어려움
- 어떤 layer 적용할지 heuristic

◆ 7. Conclusion (결론)

- LoRA:
 - 효율적 fine-tuning 방법
 - 성능 유지 + 비용 대폭 감소
- 대형 모델 시대에:
 - 👉 사실상 표준 기법

◆ 8. 기존 대비 차별점 ★

- Adapter vs LoRA:
 - Adapter: 구조 추가 (느림)
 - LoRA: 기존 weight에 병합 (빠름)
- Prompt tuning vs LoRA:
 - Prompt: 입력 제한 있음
 - LoRA: 모델 내부 직접 수정

👉 핵심 차별:

“low-rank weight update + zero latency”

◆ 9. 핵심 아이디어 한 줄 요약

👉 “fine-tuning은 필요 없고, weight 변화만 low-rank로 학습하면 된다”

2. RAG

논문 분석: Retrieval-Augmented Generation (RAG)

1. 어떤 문제를 해결하려 했는가? (Abstract)

- 기존 LLM은 지식을 파라미터 내부에만 저장
 - 문제:
 - 최신 정보 업데이트 어려움
 - 근거(provenance) 부족
 - hallucination 발생
 - 해결:
 - 외부 지식(위키피디아) + 생성 모델 결합
 - Retrieval + Generation 통합 모델 제안
-

2. 연구 동기와 문제점 (Introduction)

- LLM = implicit knowledge base (내부 기억)
 - 한계:
 - 수정 불가능 (update 어려움)
 - 설명 불가능 (왜 그런 답인지 모름)
 - 아이디어:
 - **parametric memory + non-parametric memory 결합**
-

3. 관련 연구 동향 (Related Works)

- Parametric-only: GPT, BERT, T5
- Retrieval 기반:
 - REALM, ORQA (extractive QA)
- Memory 기반:

- Memory Network, KNN-LM

👉 기존 한계:

- 특정 task에만 최적화됨
→ RAG는 범용 architecture 목표

4. 연구 접근법 & 모델 구조 (Method)

핵심 구조

- Retriever: DPR (문서 찾기)
- Generator: BART (텍스트 생성)

👉 구조

```
input x → retriever → 문서 z  
(x + z) → generator → output y
```

핵심 확률 모델

(1) RAG-Sequence

- 하나의 문서로 전체 생성

$$p(y|x) \approx \sum_z p_{\eta}(z|x) p_{\theta}(y|x, z)$$

(2) RAG-Token

- token마다 다른 문서 사용

$$p(y|x) \approx \prod_i \sum_z p_\eta(z|x) p_\theta(y_i|x, z, y_{<i})$$

👉 핵심 차이:

- Sequence: document 1개
- Token: document 여러 개

Retriever (DPR)

$$p_\eta(z|x) \propto \exp(d(z)^T q(x))$$

- BERT encoder 기반
- MIPS로 top-k 검색

학습 방식

- latent variable: z (문서)
- objective:

$$-\log p(y|x)$$

👉 retriever + generator end-to-end 학습

5. 실험 구성과 결과 (Experiments)

데이터

- Wikipedia (21M 문서)
- QA: NQ, TriviaQA, WebQ, CuratedTrec

- Generation: MS-MARCO, Jeopardy
 - Fact check: FEVER
-

결과

- QA: SOTA 달성
- Generation:
 - 더 factual
 - 더 diverse
- FEVER:
 - SOTA 근접 성능

👉 핵심 결과:

- parametric + retrieval 결합이 가장 강력
-

6. 발견 & 한계 (Discussion)

발견

- hallucination 감소
- factuality 증가
- retrieval이 성능 핵심

한계

- Wikipedia bias 존재
 - retrieval 품질 의존
 - 계산량 증가
-

7. 결론 (Conclusion)

- RAG = LLM + external memory

- 다양한 NLP task에서 강력
 - 새로운 방향:
 - hybrid memory 모델
-

8. 기존 연구 대비 차별점

기존	RAG
parametric-only	param + non-param
task-specific	범용
수정 어려움	index 교체로 업데이트 가능

👉 특히:

- retriever + generator joint training
-

9. 핵심 아이디어

👉 한 줄 요약:

| "필요할 때 외부 지식을 찾아와서 생성한다"
